LEGACY SYSTEM MODERNIZATION, REFACTORING, AND TECHNICAL DEBT MANAGEMENT

Engineering Case Study for Long-Lived Enterprise Systems

Author	Venkata Lakshmi Parimala Pasupuleti
Primary domain	Affordable Housing enterprise software
Systems represented	Income limits, documents, database migrations, Git, Docker, AWS S3, AI modernization
Document type	Technical engineering case study
Implementation boundary	Real production experience plus clearly labeled modernization guidance
Core principle	Modernize without losing business correctness, history, or operational control
Status	Portfolio documentation based on real engineering situations

This document distinguishes systems and migrations I directly owned or supported from future modernization patterns. It does not claim that I led a complete enterprise-wide rewrite or platform transformation.

Document Map

1. Executive Summary
2. Scope and Experience Boundaries
3. Why Legacy Systems Become Difficult
4. Modernization Principles
5. Technical Debt Classification
6. Prioritization and Business Value
7. Stabilization Before Refactoring
8. Characterization Tests and Baselines
9. Data and Schema Modernization
10. Document Platform and Merge-Field Migration
11. Production Database Migration
12. SQL and Stored-Procedure Refactoring
13. Source-Control Modernization
14. Docker and Environment Standardization
15. AWS S3 Document Integration
16. Service Boundaries and Microservices Decisions
17. Compatibility and Coexistence
18. Release, Rollback, and Operational Safety
19. Observability During Modernization
20. Knowledge Transfer and Adoption
21. Incident-Driven Debt Reduction
22. Security and Compliance Considerations
23. Modernizing with AI Assistance
24. Failure Modes and Anti-Patterns
25. Architecture Trade-offs
26. Modernization Roadmap
27. Reusable Checklists
28. Lessons Learned

1. Executive Summary

Long-lived enterprise systems accumulate complexity because business rules, customer workflows, data history, integrations, and operational processes evolve over many years. The resulting technical debt is not automatically evidence of poor engineering. Much of it is the cost of preserving behavior while the business changes. The engineering challenge is to distinguish valuable historical capability from accidental complexity and then modernize safely.

My experience in Affordable Housing included systems with complex stored procedures, annual data transformations, document merge fields, high-risk database migrations, SVN-based workflows, object-storage migration, and growing interest in Docker and service modernization. In each case, the modernization problem was constrained by customer continuity and production risk. A technically cleaner design had little value if it broke eligibility, financial calculations, generated documents, or historical behavior.

This case study presents a practical modernization approach: discover the real system, stabilize it with tests and evidence, introduce the new capability incrementally, run old and new behavior together where necessary, migrate callers and data in controlled phases, and retire legacy components only after production evidence supports removal.

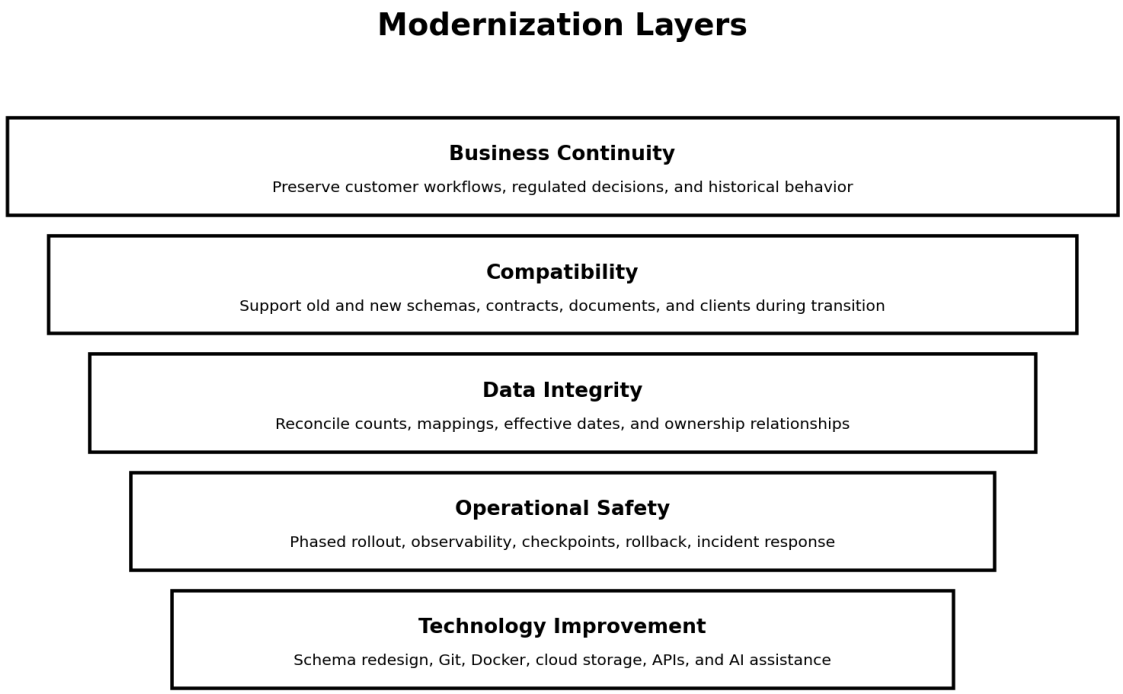


Figure 1. Technology improvement succeeds only when business continuity, compatibility, data integrity, and operational safety are preserved.

2. Scope and Experience Boundaries

The document combines direct experience from production modules with broader technical patterns I would apply to future modernization. The distinction matters because credibility depends on explaining what was implemented, what was integrated, and what was proposed.

Area	Direct experience represented	Target or broader design
Document platform	Owned Affordable Housing compatibility, merge-field mapping, validation, and urgent production defects during a schema redesign	Formal contract registry, automated template compatibility analysis, shadow rendering
Database migration	Supported a Version 2 production migration, database evidence collection, phased rollout, and dispute resolution	Automated expand-contract migrations, migration observability platform
SQL modernization	Long-term ownership of stored procedures, transformations, debugging, and performance optimization	Systematic domain-service extraction where it improves ownership and testability
Source control	Self-learned Git, led knowledge transfer, and supported SVN-to-Git adoption	Enterprise branch policy, automated compliance and repository governance
Docker	Self-directed hands-on learning and containerization of personal projects; recommended use where appropriate	Fully containerized production platform and orchestration
AWS S3	Owned Affordable Housing document integration and validation within a shared migration framework	Enterprise cloud-storage architecture and company-wide migration ownership
AI modernization	Built LeaseI and AgentProbe components and designed future extensions	Full production AI platform integrated with enterprise systems

3. Why Legacy Systems Become Difficult

Legacy difficulty usually comes from several interacting forms of coupling rather than from old syntax alone.

- Business rules are distributed across application code, stored procedures, configuration, templates, and support knowledge.
- Data structures reflect years of compatibility requirements and effective-dated history.
- Customer-created templates and documents depend on implicit merge-field contracts.
- Release processes contain manual knowledge that may not be captured in code.
- Cross-team settings can change module behavior without appearing in the module repository.
- Production reliability depends on a small number of domain experts.
- Tests may validate individual functions but not the full historical behavior of the system.

A modernization plan that treats the problem only as a framework upgrade or schema cleanup will miss these dependencies. The first responsibility is to understand the behavioral contract that the existing system already fulfills.

4. Modernization Principles

Preserve behavior before improving structure

Create evidence of current behavior before refactoring. Modernization should change structure intentionally, not business results accidentally.

Change one risk dimension at a time

Avoid changing schema, infrastructure, framework, business logic, and deployment model in one release unless the business need justifies the combined risk.

Prefer incremental migration

Introduce new paths alongside existing ones and migrate by module, property, document type, or workflow.

Keep rollback realistic

Rollback must consider code, data, queues, documents, settings, and external storage, not only application binaries.

Measure adoption and correctness

A new platform is not successful because it was deployed. It is successful when callers migrate, customers are unaffected, operations improve, and the old path can be retired.

Modernize ownership and knowledge

Technical change without documentation, runbooks, and team enablement transfers debt rather than removing it.

Controlled Modernization Pattern

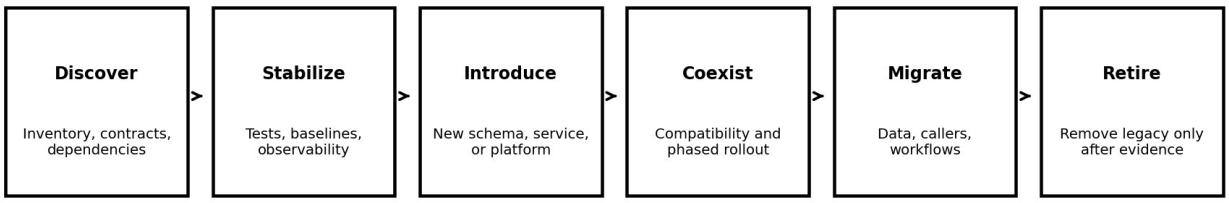


Figure 2. A controlled modernization pattern from discovery through verified legacy retirement.

5. Technical Debt Classification

Technical debt should be classified because different debt requires different treatment.

Debt type	Examples from represented systems	Typical treatment
Structural debt	Tightly coupled tables, old merge-field schema, difficult stored procedures	Refactor behind stable contracts; introduce compatibility layer
Operational debt	Manual release checks, undocumented rollback, weak alerting	Runbooks, automation, checkpoints, observability
Knowledge debt	Critical behavior understood by one module owner	Documentation, KT, ownership rotation, searchable incidents
Tooling debt	SVN workflow, inconsistent environments, manual local setup	Git adoption, Dockerized development, CI/CD
Data debt	Unclear provenance, in-place history changes, weak reconciliation	Versioning, effective dates, lineage, validation
Integration debt	Hidden system settings and cross-team dependencies	Explicit contracts, configuration governance, dependency maps
AI-readiness debt	Unstructured knowledge and limited evaluation	RAG, tools, benchmark datasets, tracing, safety gates

6. Prioritization and Business Value

Not every old component should be rewritten. Priority should reflect business impact, operational pain, recurrence, security risk, change frequency, replacement cost, and the confidence that modernization will improve outcomes.

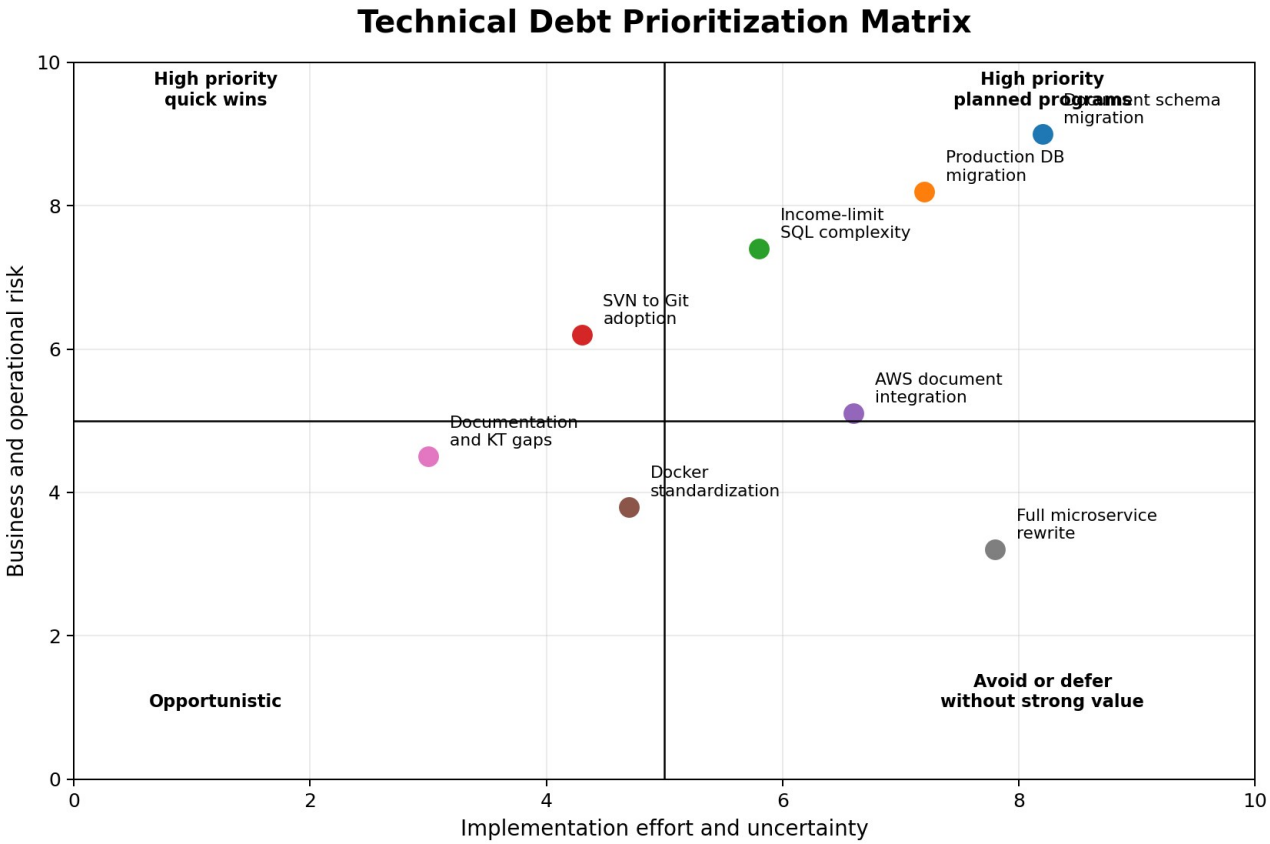


Figure 3. Technical debt is prioritized by business risk and implementation uncertainty, not by age alone.

Decision factor	Questions
Customer impact	Does the debt cause defects, delays, or inconsistent behavior?
Operational burden	How much support, release, or incident time does it consume?
Change frequency	Is the area modified often enough that current friction compounds?
Blast radius	How many modules, customers, documents, or records depend on it?
Reversibility	Can the change be rolled out and rolled back incrementally?
Knowledge concentration	Would loss of one expert create serious risk?
Strategic alignment	Does the improvement enable cloud, AI, new APIs, or faster delivery?

A full rewrite can look attractive when code is difficult, but it often creates the highest combined uncertainty. Refactoring, wrapping, and incremental replacement are usually safer when the existing system contains years of hidden business behavior.

7. Stabilization Before Refactoring

Refactoring begins with stabilization. The goal is to make current behavior observable and reproducible enough that structural change can be evaluated objectively.

1. Inventory entry points, callers, data stores, settings, scheduled jobs, queue consumers, templates, and external dependencies.
2. Identify the most business-critical workflows and representative customer scenarios.
3. Capture current outputs, record counts, execution times, and failure patterns.
4. Add targeted logging and correlation around unclear boundaries.
5. Document manual release and support procedures that currently live in team memory.
6. Fix severe data-integrity or security defects before broad redesign.
7. Establish a known production baseline against which the new path can be compared.

8. Characterization Tests and Baselines

When a system lacks comprehensive tests, characterization tests preserve current behavior before the internal design changes. They are especially valuable for stored procedures, merge fields, effective dates, and document output.

System area	Characterization approach
Income-limit transformations	Snapshot representative input and final domain records; compare counts, keys, values, and effective dates
Eligibility calculations	Capture boundary scenarios and the exact rule inputs and outcomes
Stored procedures	Run against controlled datasets and compare result sets and side effects
Document generation	Render approved templates and compare required values and output metadata
Migration scripts	Run on copies of realistic production states; verify rerun and rollback behavior
Queue processing	Replay representative messages and verify idempotent business effects

Characterization tests are not an endorsement of every legacy behavior. They identify what will change. Any intentional difference should be reviewed as a business change rather than hidden inside refactoring.

9. Data and Schema Modernization

Schema modernization is risky because application code, reports, stored procedures, migrations, documents, and support queries often depend on existing structures. The safest approach is usually expand-contract.

8. Add the new schema or columns without removing the old representation.
9. Backfill data using reviewed, observable, repeatable scripts.

10. Introduce a compatibility layer that reads or writes both representations where necessary.
11. Migrate consumers in controlled groups and compare outcomes.
12. Stop new writes to the old representation after all callers are verified.
13. Retain or archive the legacy data long enough to support rollback and investigation.
14. Remove obsolete structures only after production evidence and ownership approval.

Historical and effective-dated data requires particular care. Modernization should not flatten history into a single current value. The target design should continue to reproduce the rules and values that were active when a prior certification or document was produced.

10. Document Platform and Merge-Field Migration

The document platform migration is one of the clearest examples of modernization under compatibility pressure. Another team redesigned the merge-field database, while the Affordable Housing module still had to support existing templates and customer documents. Hundreds of documents could be generated daily, so an incorrect mapping could affect both new output and previously configured templates.

My responsibility was not the enterprise-wide schema redesign. I owned the Affordable Housing side: understanding existing document types, mapping legacy fields to the new structure, validating compatibility, supporting QA, and resolving production defects after rollout.

Modernization risk	Control
Legacy template references field removed or renamed	Compatibility mapping and deprecated-field support
New schema changes field type or source	Explicit transformation and representative document testing
Customer-specific templates behave differently	Inventory and risk-based sample selection
Incorrect migration affects many generated documents	Phased rollout, field-level telemetry, regeneration plan
Old and new document versions become hard to explain	Template, merge-field, data, and generation version metadata

The day with six urgent bugs demonstrated why migration quality cannot be evaluated only by schema success. The real contract is the rendered customer document. Modernization required end-to-end validation from field definition through data retrieval and final output.

11. Production Database Migration

The Version 2 rollout reinforced that migration work is both technical and evidentiary. When a manager believed the migration was wrong, the correct response was not to defend the script from memory. The team collected database dumps, worked with database engineers, compared states, and analyzed timestamps. The evidence showed that the migration itself was correct and that observation timing had caused confusion.

This experience shaped a reusable migration discipline:

- Capture pre-migration baselines and immutable evidence.
- Use phased rollout and explicit checkpoints.

- Record script versions, execution time, affected rows, and validators.
- Separate the time of data change from the time a problem was reported.
- Define rollback for state, not only for code.
- Treat disagreement as an investigation problem, not a personal conflict.

12. SQL and Stored-Procedure Refactoring

Stored procedures can be valuable for data-intensive transformations, transactional consistency, and performance, but they become difficult when business rules, orchestration, and presentation concerns accumulate without clear boundaries.

Keep in database when	Move or expose through application layer when
Set-based transformation is the primary responsibility	Workflow orchestration spans services, queues, or external APIs
Transaction boundaries and data locality are critical	Business behavior needs independent versioning and unit-level testing
The procedure is stable and measurable	Change frequency and ownership make database release coordination a bottleneck
SQL expresses the logic more clearly than row-by-row code	The logic depends on user context, authorization, or non-database evidence

Refactoring should begin by documenting inputs, outputs, side effects, error behavior, expected row counts, and performance baseline. Large procedures can then be separated into named stages, staging tables, validation steps, or application-controlled workflows without changing the business result in one step.

13. Source-Control Modernization

SVN-to-Git adoption was both a tooling change and a team enablement effort. The team had understandable concern because source control affects every developer and a mistake can block delivery. I learned Git independently, conducted knowledge-transfer sessions, and helped the team become comfortable with the new model.

- Preserve meaningful history, branches, and release tags where required.
- Define the new workflow in simple terms before cutover.
- Train developers on everyday recovery actions, not only ideal commands.
- Provide a transition window and support for common mistakes.
- Connect Git adoption to code review, CI/CD, and release traceability.
- Avoid treating tool resistance as a people problem when the real issue is operational risk.

Modernization succeeds when the team can operate the new system confidently. A technically correct repository conversion without adoption support simply creates a new kind of debt.

14. Docker and Environment Standardization

I was initially not part of the Docker and AWS teams, so I learned Docker independently by building and containerizing personal projects. That work gave me hands-on experience with Dockerfiles, images,

containers, Compose, ports, volumes, and service configuration. I later recommended Docker where it could improve consistency.

The direct experience should be described accurately: I did not lead a full production container migration. The modernization value I identified was reproducible local environments, dependency isolation, simpler onboarding, and a path toward consistent CI/CD artifacts.

Containerization benefit	Required discipline
Environment consistency	Versioned image, deterministic build, externalized configuration
Faster onboarding	Documented Compose setup and realistic local dependencies
Deployment portability	Health checks, immutable image, controlled secrets
Service isolation	Clear ownership, network contracts, resource limits
Safer change	Image scanning, CI validation, rollback to known artifact

Containerizing an unclear architecture does not modernize it. Docker should package well-understood boundaries; it should not hide coupling or justify premature microservices.

15. AWS S3 Document Integration

The AWS document migration separated shared migration-framework ownership from module integration ownership. I owned the Affordable Housing portion: document classification, framework adaptation, mapping, validation, and QA support. This scope demonstrates an important modernization skill - integrating a complex domain into a platform capability without overstating ownership of the platform itself.

The migration had to preserve content, metadata, ownership, retrieval behavior, and customer workflows. A successful object copy was not sufficient. The application needed to locate the correct document, enforce the same access path, and maintain enough metadata for support and rollback.

16. Service Boundaries and Microservices Decisions

Microservices are not the default definition of modernization. A service boundary is justified when it improves ownership, scalability, release independence, failure isolation, or security enough to offset network, data, observability, and operational complexity.

Candidate boundary	Reason it may be useful	Reason to keep together
Document generation	Independent workload, storage, rendering, and scaling	Tight coupling to templates and domain data may make early separation expensive
Annual data ingestion	Batch lifecycle, validation, specialized ownership	Low frequency may not justify a continuously operated service

Candidate boundary	Reason it may be useful	Reason to keep together
Eligibility engine	Clear regulated responsibility and deterministic contract	Requires strong consistency with certification and configuration data
AI investigation	Different runtime, model dependencies, and evaluation needs	Should remain read-only and integrated through controlled tools

The correct modernization question is not "How do we break the monolith?" It is "Which boundary reduces business and operational friction while preserving correctness?"

17. Compatibility and Coexistence

Most enterprise modernization requires a period where old and new systems coexist. Compatibility strategies include adapters, dual reads, dual writes, shadow execution, versioned APIs, feature flags, and phased tenant migration.

Strategy	Use	Risk
Adapter	Translate old contract to new implementation	Adapter can become permanent hidden complexity
Dual read	Compare old and new data paths	May double load and create conflict-resolution questions
Dual write	Keep both stores updated	Creates consistency and transaction risk
Shadow execution	Run new logic without affecting outcome	Requires reliable comparison and added cost
Versioned API	Allow consumers to migrate independently	Extends support window and contract surface
Feature flag	Control rollout by customer or workflow	Flag lifecycle and configuration drift must be governed

Compatibility should have an exit plan. Temporary bridges need owners, retirement criteria, and monitoring; otherwise modernization creates another permanent layer.

18. Release, Rollback, and Operational Safety

Modernization increases release risk because multiple representations and workflow paths may exist at the same time. Release planning should identify the customer-visible contract, migration order, checkpoints, fallback path, and evidence required to continue.

- Deploy backward-compatible schema before dependent application changes.
- Separate data backfill from the final cutover where possible.
- Use feature flags or tenant cohorts for controlled activation.
- Monitor old and new outcomes during coexistence.

- Define how queued work, documents, caches, and external objects behave during rollback.
- Pause or stop rollout when evidence becomes ambiguous.

19. Observability During Modernization

Migration telemetry should make both progress and correctness visible.

Signal area	Examples
Coverage	Callers migrated, documents converted, rows backfilled, tenants activated
Comparison	Old/new result difference, missing mappings, field mismatch, output checksum
Performance	Latency and resource comparison between legacy and new paths
Reliability	Retries, failed conversions, rollback events, compatibility fallbacks
Adoption	Legacy path traffic, new path traffic, remaining dependencies
Retirement	Zero-use period, archived evidence, removed flags and adapters

A new component should not replace the legacy path until the system can show that the new path is correct, supportable, and actually used.

20. Knowledge Transfer and Adoption

Technical debt includes knowledge concentration. I became a long-term owner of difficult modules because I understood the stored procedures, annual cycles, production behavior, and business consequences. That ownership was valuable, but it also demonstrated why documentation and knowledge transfer are necessary.

- Document system purpose, data flow, dependencies, settings, release lifecycle, and common failures.
- Pair modernization work with domain knowledge transfer.
- Use code review and design review to spread context, not only enforce style.
- Create operational runbooks that another engineer can execute.
- Capture confirmed incidents as searchable organizational knowledge.
- Measure whether the team can operate the new system without the original owner.

21. Incident-Driven Debt Reduction

Production incidents expose the most expensive technical debt. The correct response is not always a rewrite. The objective is to identify the smallest durable change that reduces recurrence and investigation time.

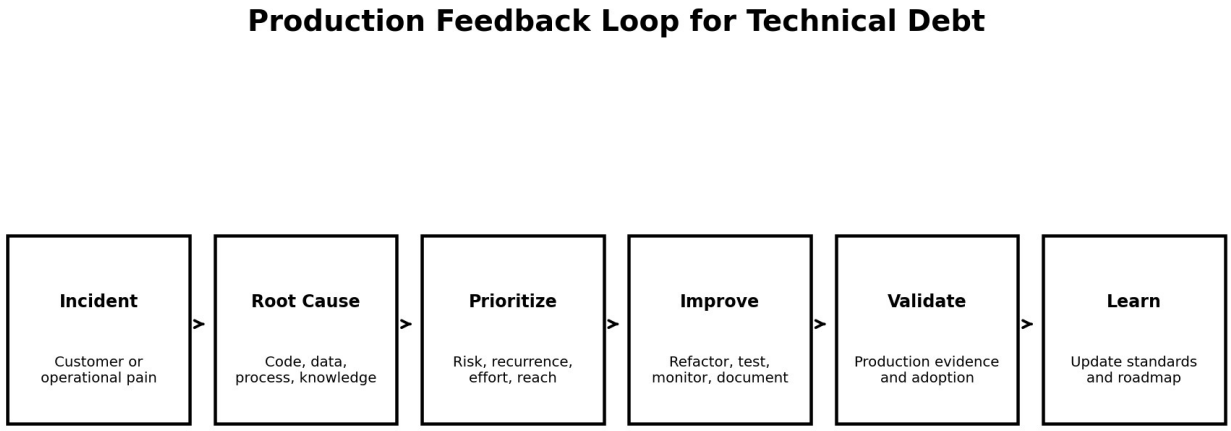


Figure 4. Incidents become modernization inputs when root causes are translated into durable engineering changes.

Incident lesson	Durable modernization response
Hidden property setting caused ineligibility	Configuration inventory, dependency contract, audit history, LeaseI retrieval
Migration correctness disputed	Preserved evidence, checkpoint telemetry, standardized validation
Six urgent document defects	Field-level compatibility tests, migration inventory, better generation diagnostics
Complex annual import debugging	Stage-level reconciliation, explicit workflow state, reusable runbook
Team fear of Git	Training, transition support, documented recovery workflows

22. Security and Compliance Considerations

Modernization must not weaken tenant isolation, historical auditability, access control, or regulated decision boundaries. Moving data to a new schema, storage platform, queue, container, or AI system creates new trust boundaries.

- Preserve authorization and ownership metadata during data and document migration.
- Avoid broadening production credentials for convenience during transition.
- Retain source versions, migration evidence, and historical effective dates.
- Minimize sensitive data in queues, logs, container images, and AI prompts.
- Keep LeaseI-style diagnostics read-only until write actions have explicit approval and controls.
- Test cross-tenant and rollback behavior as part of the migration, not after it.

23. Modernizing with AI Assistance

AI can reduce the knowledge and investigation burden around legacy systems when it is grounded in trusted evidence. LeaseI is designed to combine product knowledge, regulations, historical incidents, and live structured tools. In a modernization program, that architecture could help engineers discover dependencies, explain configuration, compare incidents, and create better escalation packets.

AI should not independently rewrite or migrate regulated production logic. Safer uses include documentation synthesis, code and schema exploration, test generation for human review, log summarization, incident similarity search, and evidence-linked support assistance. AgentProbe-style evaluation is needed before AI-generated changes influence production decisions.

24. Failure Modes and Anti-Patterns

Anti-pattern	Why it fails
Big-bang rewrite	Combines unknown business behavior with maximum technical and operational uncertainty
Framework-first modernization	Changes technology without resolving ownership, data, testing, or workflow problems
Dual write without consistency design	Creates silent divergence between old and new stores
Compatibility without retirement plan	Turns temporary adapters and flags into permanent complexity
Migration without evidence	Leaves teams unable to prove correctness or explain incidents
Containerize everything	Packages poor boundaries and adds operational overhead without business value
Microservices by table	Creates network coupling and distributed transactions without meaningful ownership gains
Ignore team adoption	Produces a technically deployed platform that engineers avoid or misuse

25. Architecture Trade-offs

Refactor versus rewrite

Refactoring preserves behavior and supports incremental value. Rewrites can simplify architecture but concentrate risk and delay feedback.

Database logic versus application logic

Database logic provides data locality and set-based performance. Application logic may improve modularity, versioning, and testability.

Compatibility versus simplicity

Compatibility protects customers during transition but increases temporary code paths and operational burden.

Standardization versus autonomy

Shared platforms reduce duplication, while domain teams need flexibility for specialized requirements.

Immediate cleanup versus evidence retention

Removing legacy data quickly simplifies the system, but retaining it temporarily supports rollback and investigation.

Modern platform versus actual scale

Cloud, queues, containers, and vector databases should be introduced for real operational requirements, not portfolio appearance.

26. Modernization Roadmap

Phase	Focus	Outcome
1. Discover	Inventory behavior, dependencies, owners, data, settings, and operational pain	A factual modernization baseline
2. Stabilize	Characterization tests, observability, runbooks, critical defect fixes	Change becomes measurable and safer
3. Establish contracts	APIs, schemas, document fields, events, configuration precedence	Clear boundaries between old and new
4. Introduce target components	New schema, Git workflow, containerized service, cloud storage, AI assistant	Modern capability exists without forced cutover
5. Migrate incrementally	Backfill, cohorts, shadow comparison, controlled consumer migration	Business risk remains bounded
6. Validate operations	Performance, support, security, rollback, adoption, and ownership	Target system becomes production-ready
7. Retire legacy	Remove unused code, flags, adapters, and data after evidence	Complexity and operating cost actually decline

27. Reusable Checklists

27.1 Before modernization

- Identify the business contract and highest-risk workflows.
- Inventory callers, data, settings, documents, queues, jobs, and owners.
- Capture production baseline and representative outputs.
- Document explicit scope and ownership boundaries.
- Classify debt by business risk and recurrence.

27.2 During migration

- Use backward-compatible change where possible.
- Record coverage, comparison results, performance, and failures.
- Keep checkpoints, rollback, and incident evidence available.
- Control rollout with cohorts or flags and govern temporary compatibility.
- Communicate status, risk, and decision criteria across teams.

27.3 Before retirement

- Verify all callers and data are migrated.
- Observe a defined zero-use period on the legacy path.
- Confirm runbooks, ownership, monitoring, and support readiness for the new path.
- Archive required history and remove sensitive obsolete data according to policy.
- Remove flags, adapters, credentials, and operational dashboards that no longer have purpose.

28. Lessons Learned

- Legacy systems contain business history, not only old code.
- Modernization succeeds through evidence and sequencing, not through technology choice alone.
- The safest migration changes structure while preserving behavior and customer continuity.
- Compatibility is a temporary product capability that needs ownership and retirement criteria.
- Production incidents reveal where technical debt has the highest real cost.
- Tool adoption requires team enablement, not only technical conversion.
- Long-term ownership creates expertise, but modernization should distribute that knowledge.
- Microservices and containers are useful when they improve ownership and operations, not as goals by themselves.
- AI can accelerate understanding and investigation, but regulated production decisions remain deterministic and human-accountable.
- Scope accuracy is part of senior engineering credibility: integration ownership should not be overstated as platform ownership.

Closing Perspective

My modernization approach is shaped by years of operating business-critical systems. I do not see legacy code as something to discard simply because it is old. I see it as a set of customer and business contracts that must be understood before they can be improved. The senior engineering responsibility is to reduce complexity while preserving correctness, history, supportability, and trust.